



PERTEMUAN III

Classification

3.1. TUJUAN PEMBELAJARAN

- A. Mahasiswa mampu memahami konsep dasar beberapa algoritma klasifikasi, seperti *Logistic Regression*, KNN, *Decision Tree*, dan *Random Forest*
- B. Mahasiswa mengetahui fungsi dan peran metrik evaluasi klasifikasi, seperti *accuracy*, *precision*, dan *recall* dalam menilai performa model
- C. Mahasiswa dapat menerapkan berbagai algoritma klasifikasi untuk membangun model
- D. Mahasiswa mampu melakukan evaluasi model dengan menyusun tabel performa berisi *accuracy*, *precision*, dan *recall* dari setiap model

3.2. DASAR TEORI

- **Classification**

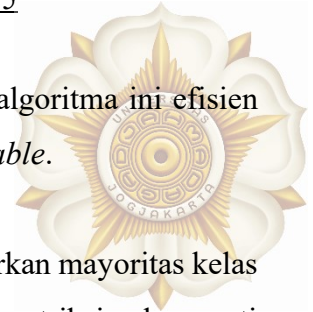
Klasifikasi merupakan salah satu teknik *supervised learning* yang bertujuan untuk memprediksi label atau kategori dari sebuah sampel berdasarkan fitur yang dimiliki. Model klasifikasi belajar dari data berlabel, kemudian menghasilkan suatu fungsi yang mampu menentukan kelas dari data baru yang belum pernah dilihat. Klasifikasi bekerja dengan cara mencari pola dari data historis untuk menentukan keputusan kelas yang paling mendekati. Klasifikasi umum digunakan dalam aplikasi seperti deteksi spam, diagnosis medis, hingga rekomendasi konten.

- **Algoritma Classification**

Ada banyak metode yang bisa dipakai untuk melakukan klasifikasi. Beberapa algoritma di bawah ini termasuk yang paling sering digunakan dan dijelaskan dalam dokumentasi Scikit-Learn dan TensorFlow karena sederhana, efektif, dan cocok untuk berbagai jenis dataset.

1. *Logistic Regression*

Logistic Regression merupakan algoritma klasifikasi berbasis regresi linier yang memodelkan probabilitas suatu sampel termasuk ke dalam kelas tertentu menggunakan fungsi *sigmoid*. *Output* model berada dalam rentang 0–1, yang



kemudian dikonversi menjadi label kelas. Meskipun sederhana, algoritma ini efisien dan bekerja baik pada data yang memiliki hubungan linear *separable*.

2. *K-Nearest Neighbors (KNN)*

K-Nearest Neighbors mengklasifikasikan suatu data baru berdasarkan mayoritas kelas dari k tetangga terdekatnya. Kedekatan dihitung menggunakan metrik jarak seperti *Euclidean distance*. Algoritma ini bersifat *non-parametric* dan sangat bergantung pada struktur distribusi data serta nilai k yang dipilih.

3. *Decision Tree*

Decision Tree bekerja dengan cara membagi dataset secara rekursif berdasarkan fitur yang memberikan informasi terbaik. Proses pemilihan fitur dilakukan menggunakan ukuran seperti *Gini impurity* atau *entropy/information gain*. Kelebihannya adalah interpretasi yang mudah, tetapi mudah mengalami *overfitting* tanpa pengaturan parameter yang tepat

4. *Random Forest*

Random Forest adalah algoritma *ensemble* yang menggabungkan banyak *Decision Tree* melalui teknik *bagging*. Setiap pohon dilatih pada subset data yang berbeda, dan hasil akhirnya ditentukan melalui voting. *Random Forest* membuat model lebih stabil, mengurangi *overfitting*, dan meningkatkan akurasi dibanding satu pohon tunggal.

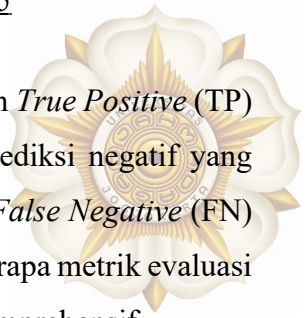
5. *Adaboost (Adaptive Boosting)*

AdaBoost bekerja dengan menggabungkan beberapa *weak learner* (biasanya *Decision Tree* kecil) secara berurutan. Pada setiap iterasi, bobot sampel diperbarui sehingga model berikutnya lebih fokus pada data yang sebelumnya salah diklasifikasikan. Dengan cara ini, *AdaBoost* mampu meningkatkan performa model secara signifikan terutama pada dataset yang kompleks.

• **Metrik Evaluasi Classification**

Dalam *machine learning* khususnya pada model klasifikasi, diperlukan metrik evaluasi untuk menilai seberapa baik model mampu memprediksi label dengan benar. Evaluasi ini dilakukan dengan membandingkan hasil prediksi model terhadap nilai aktual melalui sebuah struktur yang disebut *confusion matrix*.

1. *Confusion Matrix*



Confusion matrix terdiri dari empat komponen penting, antara lain *True Positive* (TP) yaitu prediksi positif yang benar, *True Negative* (TN) yaitu prediksi negatif yang benar, *False Positive* (FP) yaitu prediksi positif yang salah, dan *False Negative* (FN) yaitu prediksi negatif yang salah. Berdasarkan nilai-nilai ini, beberapa metrik evaluasi dapat dihitung untuk memahami performa model secara lebih komprehensif.

2. *Accuracy, Precision, Recall, F1 Score*

Accuracy merupakan metrik paling dasar yang menunjukkan proporsi prediksi yang benar dari seluruh prediksi yang dibuat oleh model. Rumusnya didefinisikan sebagai berikut:

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

Precision mengukur tingkat ketepatan model dalam melakukan prediksi positif, yaitu berapa banyak prediksi positif yang benar-benar positif. *Precision* dihitung dengan rumus berikut:

$$Precision = \frac{TP}{TP + FP}$$

Sementara itu, *Recall* mengukur kemampuan model dalam menemukan seluruh sampel yang benar-benar positif. *Recall* dihitung menggunakan rumus:

$$Recall = \frac{TP}{TP + FN}$$

Karena *precision* dan *recall* memiliki fokus yang berbeda, F1-Score digunakan sebagai metrik yang menyeimbangkan keduanya. Metrik ini sangat berguna ketika dataset bersifat *imbalanced* dan diperlukan keseimbangan antara kemampuan model menghindari *false positives* serta *false negatives*. F1-score merupakan *harmonic mean* dari *precision* dan *recall*, dengan rumus:

$$F1\ Score = 2 \times \frac{Precision \times Recall}{Precision + Recall}$$

3.3. ALAT DAN BAHAN

- **Perangkat Keras**
 - Komputer/Laptop
- **Perangkat Lunak**



- Python3
- Google Colaboratory

3.4. LANGKAH PERCOBAAN

[Isi dengan instruksi langkah demi langkah yang jelas disertai penjelasan singkat.

Tambahkan gambar ilustrasi jika diperlukan]

1. Impor Pustaka

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.metrics import accuracy_score
from sklearn.metrics import precision_score
from sklearn.metrics import recall_score
from sklearn.metrics import confusion_matrix
from sklearn.metrics import ConfusionMatrixDisplay
from imblearn.metrics import sensitivity_specificity_support
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import f1_score
from sklearn.neighbors import KNeighborsClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.ensemble import AdaBoostClassifier
```

Gambar 3.4.1 Import Pustaka

2. Load dataset

Dataset dapat diunduh pada link berikut:

<https://www.kaggle.com/datasets/fedesoriano/stroke-prediction-dataset>

```
df = pd.read_csv('https://raw.githubusercontent.com/ganjar87/data_science_practice/refs/heads/main/healthcare-dataset-stroke-data.csv')
```

3. Exploratory Data Analysis (EDA)

- Melihat lima baris pertama dan terakhir pada dataset dengan `df.head()` dan `df.tail()`.

df.head()												
	id	gender	age	hypertension	heart_disease	ever_married	work_type	Residence_type	avg_glucose_level	bmi	smoking_status	stroke
0	9046	Male	67.0	0	1	Yes	Private	Urban	228.69	36.6	formerly smoked	1
1	51676	Female	61.0	0	0	Yes	Self-employed	Rural	202.21	NaN	never smoked	1
2	31112	Male	80.0	0	1	Yes	Private	Rural	105.92	32.5	never smoked	1
3	60182	Female	49.0	0	0	Yes	Private	Urban	171.23	34.4	smokes	1
4	1665	Female	79.0	1	0	Yes	Self-employed	Rural	174.12	24.0	never smoked	1

Gambar 3.4.2 Lima Baris Pertama Dataset

```
df.tail()
```

	id	gender	age	hypertension	heart_disease	ever_married	work_type	Residence_type	avg_glucose_level	bmi	smoking_status	stroke
5105	18234	Female	80.0	1	0	Yes	Private	Urban	83.75	NaN	never smoked	0
5106	44873	Female	81.0	0	0	Yes	Self-employed	Urban	125.20	40.0	never smoked	0
5107	19723	Female	35.0	0	0	Yes	Self-employed	Rural	82.99	30.6	never smoked	0
5108	37544	Male	51.0	0	0	Yes	Private	Rural	166.29	25.6	formerly smoked	0
5109	44679	Female	44.0	0	0	Yes	Govt_job	Urban	85.28	26.2	Unknown	0

Gambar 3.4.3 Lima Baris Terakhir Dataset

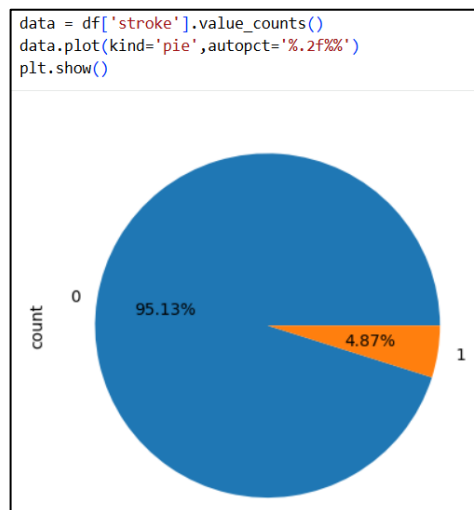
b. Melihat deskripsi statistik dengan df.describe()

```
df.describe()
```

	id	age	hypertension	heart_disease	avg_glucose_level	bmi	stroke
count	5110.000000	5110.000000	5110.000000	5110.000000	5110.000000	4909.000000	5110.000000
mean	36517.829354	43.226614	0.097456	0.054012	106.147677	28.893237	0.048728
std	21161.721625	22.612647	0.296607	0.226063	45.283560	7.854067	0.215320
min	67.000000	0.080000	0.000000	0.000000	55.120000	10.300000	0.000000
25%	17741.250000	25.000000	0.000000	0.000000	77.245000	23.500000	0.000000
50%	36932.000000	45.000000	0.000000	0.000000	91.885000	28.100000	0.000000
75%	54682.000000	61.000000	0.000000	0.000000	114.090000	33.100000	0.000000
max	72940.000000	82.000000	1.000000	1.000000	271.740000	97.600000	1.000000

Gambar 3.4.4 Hasil Deskripsi Statistik Dataset

c. Melihat presentase target kelas



Gambar 3.4.5 Presentase Target Kelas

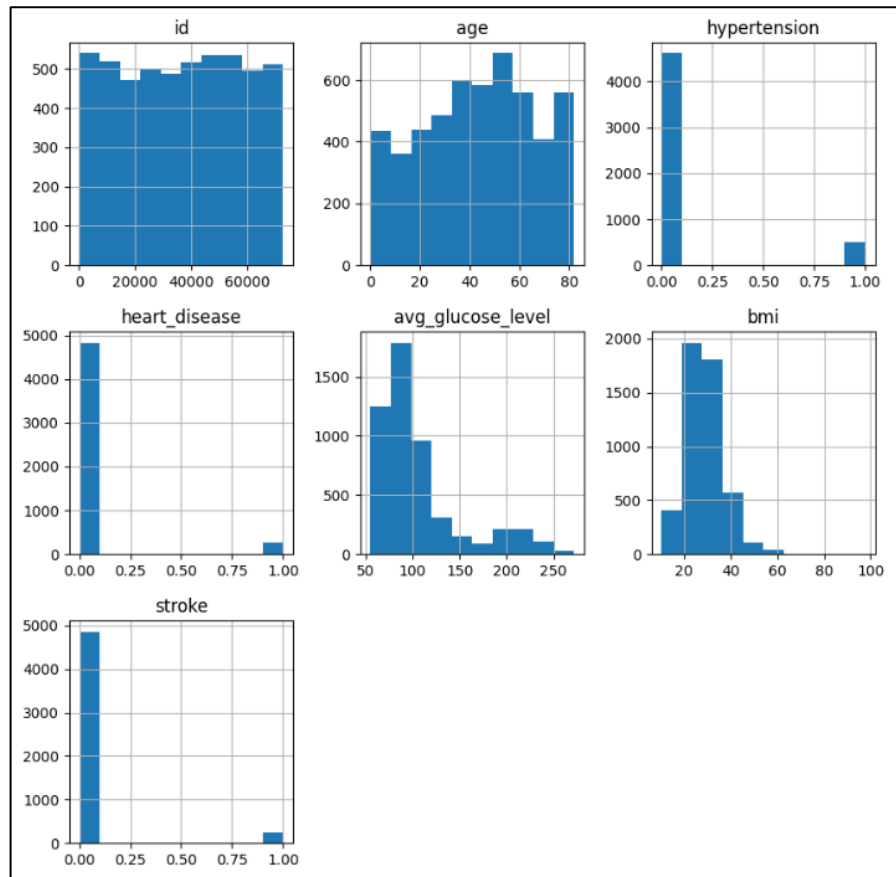
Pie chart tersebut menunjukkan distribusi variabel stroke, di mana kelas tidak mengalami stroke (0) mendominasi dengan proporsi sekitar 95.13%, sedangkan kelas mengalami stroke (1) hanya sebesar 4.87%. Hal ini menandakan bahwa dataset bersifat

imbalanced, karena jumlah sampel pada kelas positif jauh lebih sedikit dibanding kelas negatif.

d. Melihat histogram

```
df.hist(figsize=(10,10))
plt.show()
```

Gambar 3.4.6 Kode untuk Menampilkan Histogram



Gambar 3.4.7 Histogram Seluruh Variabel Numerik

e. Mengecek *missing value* dengan `df.isnull().sum()`



df.isnull().sum()	
	0
id	0
gender	0
age	0
hypertension	0
heart_disease	0
ever_married	0
work_type	0
Residence_type	0
avg_glucose_level	0
bmi	201
smoking_status	0
stroke	0
dtype: int64	

Gambar 3.4.8 Pengecekan Missing Value

Hasil pengecekan *missing value* di atas menunjukkan bahwa hanya variabel *bmi* memiliki nilai kosong sebanyak 201 baris, sementara kolom lainnya tidak memiliki *missing value*.

f. Mengecek atribut kategorikal

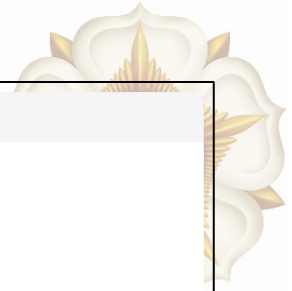
```
df_x = df.drop(['id', 'stroke'],axis=1)
# definisikan kolom yg jadi input
df_y = df[['stroke']]
# definisikan kolom yg jadi output
cats = df_x.select_dtypes(include=['object', 'bool']).columns
print(cats)

Index(['gender', 'ever_married', 'work_type', 'Residence_type',
       'smoking_status'],
      dtype='object')
```

Gambar 3.4.9 Pengecekan Atribut Kategorikal

Hasil pengecekan atribut kategorikal menunjukkan bahwa variabel kategorikal yang perlu dilakukan *encoding* yaitu *gender*, *ever_married*, *work_type*, *Residence_type*, dan *smoking_status*.

4. Data Preprocessing



```
# data preprocessing dimulai
# membuat X and y. X untuk input variable, y untuk target class
df_X = df.drop(['id', 'stroke'],axis=1)
df_y = df[['stroke']]

#label encoding for y.
#merubah nilai yg ada di y menjadi 0 atau 1.
#sebenarnya ini tidak diperlukan karena nilai y di dataset sudah 0 atau 1
le = LabelEncoder()
df_y= le.fit_transform(df_y['stroke'])

#imputation
df_X['bmi'].fillna(df_X['bmi'].median(), inplace=True)

#categorical encoding
#merubah categorical value menjadi numerical value
#bisa pakai label encoding, ordinal atau one hot encoding
cats = df_X.select_dtypes(include=['object', 'bool']).columns
cat_features = list(cats.values)
le = LabelEncoder()
for i in cat_features:
    df_X[i] = le.fit_transform(df_X[i])

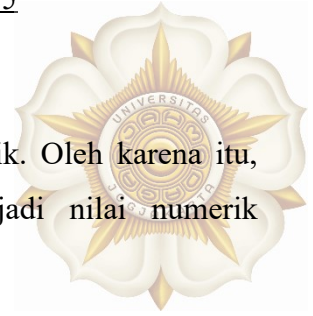
#menyimpan X dan y menjadi numpy arrays
X = df_X.astype(float).values
y = df_y.astype(float)

#hold-out method
#dibagi menjadi training dan testing set
#70% training, 30% testing
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

#scaling
scaler = StandardScaler().fit(X_train)
X_train = scaler.transform(X_train)
X_test = scaler.transform(X_test)
```

Gambar 3.4.10 Kode untuk Melakukan Data Preprocessing

- Membuat variable `df_X` untuk input variable dan `df_y` untuk target class
Pada tahap awal, dataset dipisahkan menjadi dua, yaitu variabel fitur dan variabel target. Fitur (X) merupakan atribut independen yang digunakan sebagai inputan model, sedangkan target (y) adalah variabel dependen yang ingin diprediksi.
- Label encoding* untuk y
Proses *label encoding* dilakukan untuk memastikan variabel y berada dalam format numerik.
- Imputation* dengan nilai tengah (median)
Nilai hilang pada fitur numerik diatasi menggunakan metode pengisian median, karena median lebih stabil terhadap keberadaan *outlier*.



d. *Categorical encoding*

Model *machine learning* hanya dapat memproses data numerik. Oleh karena itu, semua fitur bertipe *object* atau *boolean* dikonversi menjadi nilai numerik menggunakan Label Encoding.

e. Menyimpan x dan y menjadi *numpy arrays*

Agar kompatibel dengan fungsi-fungsi pada pustaka Scikit-Learn, variabel fitur dan target dikonversi menjadi array NumPy bertipe float.

f. Membagi data *training* dan *testing*

Dataset dibagi menjadi data *training* dan data *testing* dengan komposisi 70% data *training* dan 30% data *testing*. Pembagian ini dilakukan agar model dapat dievaluasi menggunakan data yang tidak pernah digunakan selama proses pelatihan.

g. Normalisasi data (*standardization*)

Standardisasi dilakukan menggunakan StandardScaler, yang mengubah distribusi setiap fitur sehingga memiliki nilai rata-rata 0 dan standar deviasi 1. Proses fit hanya dilakukan pada data *training* untuk mencegah data *leakage*, kemudian digunakan untuk mentransformasi data *training* dan data *testing*.

Tahapan *preprocessing* di atas memastikan bahwa dataset berada dalam kondisi optimal sebelum digunakan untuk membangun model klasifikasi. Langkah berikutnya, dilakukan proses pemeriksaan hasil *preprocessing* untuk memastikan seluruh transformasi sudah sesuai.

a. Variabel input X

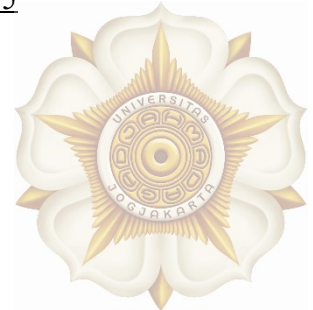
Seluruh nilai kategorik telah dikonversi menjadi numerik.

x									
array([[1. , 67. , 0. , ..., 228.69, 36.6 , 1.],									
[0. , 61. , 0. , ..., 202.21, 28.1 , 2.],									
[1. , 80. , 0. , ..., 105.92, 32.5 , 2.],									
...,									
[0. , 35. , 0. , ..., 82.99, 30.6 , 2.],									
[1. , 51. , 0. , ..., 166.29, 25.6 , 1.],									
[0. , 44. , 0. , ..., 85.28, 26.2 , 0.]])									

Gambar 3.4.11 Variabel X Setelah Dikonversi Menjadi Numerik

b. Variabel target y

Variabel target atau label berupa nilai biner (0 dan 1) hasil dari proses *label encoding*.



```
y
array([1., 1., 1., ..., 0., 0., 0.])
```

Gambar 3.4.12 Variabel *y* Setelah Proses Label Encodingc. Data *training* yang telah dinormalisasi (X_train)

X_train menampilkan nilai fitur yang telah distandardisasi, sehingga setiap kolom memiliki rata-rata 0 dan standar deviasi 1.

```
X_train
array([[ 1.18418048, -1.7467638, -0.31719928, ..., -0.340693,
        -1.64591683, -1.29622579],
       [ 1.18418048, -0.63635252, -0.31719928, ...,  2.26654137,
        -0.78843307,  1.52066342],
       [ 1.18418048,  0.02989425,  3.15259225, ..., -0.32155489,
        -0.30772248,  0.58170035],
       ...,
       [-0.84446587, -1.87290652, -0.31719928, ..., -0.18803315,
        -1.43804198, -1.29622579],
       [ 1.18418048,  1.62888649, -0.31719928, ...,  2.01062472,
        0.27692553, -0.35726272],
       [-0.84446587,  0.11872715, -0.31719928, ..., -0.12416526,
        2.78441591,  1.52066342]])
```

Gambar 3.4.13 Hasil Transformasi pada Data Training

d. Data testing yang telah dinormalisasi (X_test)

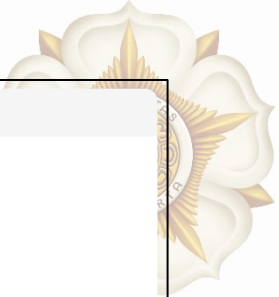
X_test menunjukkan hasil transformasi fitur pada data testing menggunakan *scaler* yang sama seperti data *training*, sehingga menghasilkan distribusi nilai yang serupa dengan angka-angka pada skala terstandarisasi.

```
X_test
array([[ 1.18418048, -0.54751962, -0.31719928, ..., -0.90971812,
        -0.76244872, -1.29622579],
       [ 1.18418048, -0.14777156, -0.31719928, ..., -0.89992653,
        -0.07386327,  0.58170035],
       [-0.84446587, -1.569098, -0.31719928, ..., -0.69675096,
        -0.82740961, -1.29622579],
       ...,
       [ 1.18418048, -0.05893866, -0.31719928, ..., -0.26569829,
        -0.21677723,  0.58170035],
       [-0.84446587,  0.60730811, -0.31719928, ..., -0.80846414,
        -0.63252693, -1.29622579],
       [-0.84446587,  0.74055746, -0.31719928, ..., -0.72746095,
        -0.46362862,  0.58170035]])
```

Gambar 3.4.14 Hasil Transformasi pada Data Testing

5. Modelling

a. Logistic Regression



```

model=LogisticRegression()
model.fit(X_train, y_train)

# membuat prediksi
y_pred = model.predict(X_test)

# y_pred jika di print out keluar vector prediksi nya
#menghitung performa model, dengan accuracy dll
print('Accuracy ',accuracy_score(y_test, y_pred))
print('Precision ',precision_score(y_test, y_pred, average='macro'))
print('Recall ',recall_score(y_test, y_pred, average='macro'))
print('Confusion matrix ', confusion_matrix(y_test, y_pred))

# plot_confusion_matrix(model, X_test, y_test, cmap=plt.cm.Blues)
cm = confusion_matrix(y_test, y_pred)
disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=np.unique(y_test))

# plt.show()
disp.plot(cmap=plt.cm.Blues)

print('F1 ',f1_score(y_test, y_pred, average='macro'))

```

Gambar 3.4.15 Kode untuk Logistic Regression

- Melatih model

Pada tahap ini, objek **LogisticRegression()** dibuat sebagai model klasifikasi. Model kemudian dilatih menggunakan data *training* (*X_train* dan *y_train*) agar dapat mempelajari hubungan antara fitur dan label target.

- Membuat prediksi

Model yang telah dilatih digunakan untuk memprediksi kelas dari data testing (*X_test*). Hasil prediksi disimpan pada variabel *y_pred*, yang berupa vektor berisi nilai kelas hasil prediksi.

- Mengukur performa model

Performa model diukur dengan metrik *accuracy*, *precision*, *recall*, dan *confusion matrix*. Penggunaan parameter **average='macro'** memastikan bahwa setiap kelas memiliki kontribusi yang sama dalam perhitungan *precision* dan *recall*.

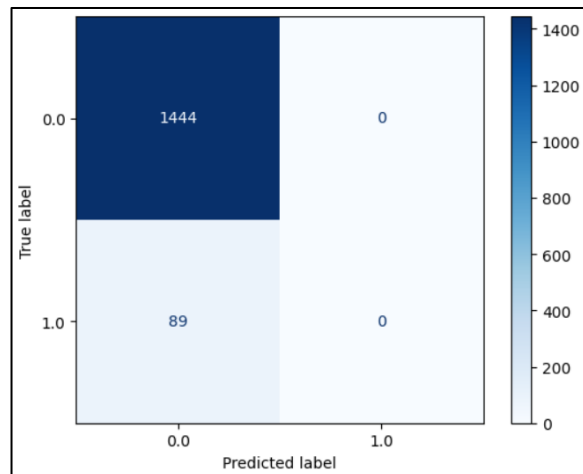
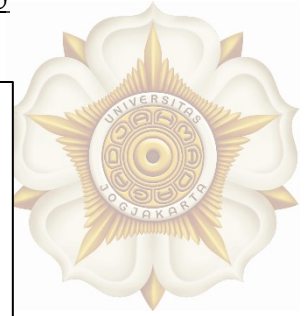
```

Accuracy 0.9419439008480104
Precision 0.4709719504240052
Recall 0.5
Confusion matrix [[1444  0]
 [ 89  0]]

```

Gambar 3.4.16 Output Performa Model

- Visualisasi *confusion matrix*



Gambar 3.4.17 Confusion Matrix

Confusion matrix divisualisasikan menggunakan warna (heatmap) untuk memudahkan interpretasi. Visualisasi ini membantu melihat pola kesalahan model, misalnya apakah model cenderung salah pada kelas tertentu.

- Menghitung *F1 score*

F1-score merupakan kombinasi harmonis dari *precision* dan *recall*, sehingga memberikan gambaran performa yang lebih seimbang terutama pada dataset dengan distribusi kelas yang tidak merata.

F1 0.485052065838092

Gambar 3.4.18 Output F1 Score

- Model *coefficient*

```
model.coef_
array([[ -0.01878829,  1.55249921,  0.10370412,  0.07921647, -0.18371621,
        -0.06959651,  0.06033662,  0.19742689, -0.03156246,  0.02218138]])
```

Gambar 3.4.19 Pengecekan Nilai Koefisien dari Model Logistic Rgression

Output `model.coef_` menunjukkan nilai koefisien yang diperoleh model *Logistic Regression* untuk setiap fitur setelah proses pelatihan. Nilai positif menunjukkan bahwa fitur tersebut cenderung meningkatkan peluang prediksi kelas 1, sedangkan nilai negatif menurunkan peluang tersebut

- Model *intercept*



```
model.intercept_  
array([-4.04531553])
```

Gambar 3.4.20 Pengecekan Nilai Konstanta pada Model Logistic Regression

Output `model.intercept_` merupakan nilai konstanta dalam fungsi keputusan model. *Intercept* bernilai *negative* menunjukkan bahwa, ketika seluruh fitur bernilai nol (setelah melalui proses normalisasi), model memiliki kecenderungan awal untuk memprediksi kelas 0.

b. *K-Nearest Neighbors*



```
model=KNeighborsClassifier(n_neighbors=10)  
model.fit(X_train, y_train)  
  
# membuat prediksi  
y_pred = model.predict(X_test)  
  
#menghitung performa model, dengan accuracy dll  
print('Accuracy ',accuracy_score(y_test, y_pred))  
print('Precision ',precision_score(y_test, y_pred, average='macro'))  
print('Recall ',recall_score(y_test, y_pred, average='macro'))  
print('Confusion matrix ', confusion_matrix(y_test, y_pred))  
  
# plot_confusion_matrix(model, X_test, y_test, cmap=plt.cm.Blues)  
cm = confusion_matrix(y_test, y_pred)  
disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=np.unique(y_test))  
  
# plt.show()  
disp.plot(cmap=plt.cm.Blues)
```

Gambar 3.4.21 Kode untuk Melatih Model KNN

- Melatih model

Model KNN dibentuk dengan menetapkan jumlah tetangga (*k*) sebanyak 10, yang berarti keputusan klasifikasi akan ditentukan berdasarkan 10 sampel terdekat pada fitur. Data training (*X_train* dan *y_train*) digunakan untuk menyimpan representasi data sehingga model dapat menghitung jarak antar sampel pada tahap prediksi.

- Membuat prediksi

Model kemudian digunakan untuk memprediksi kelas dari data testing (*X_test*). Proses prediksi dilakukan dengan menghitung jarak antara sampel testing dan seluruh sampel *training*, kemudian memilih kelas mayoritas dari 10 tetangga terdekat.

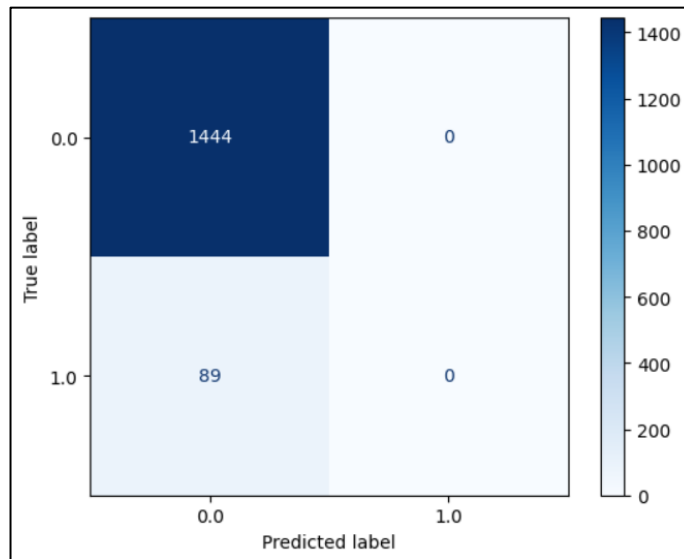


- Mengukur performa model

```
Accuracy 0.9419439008480104
Precision 0.4709719504240052
Recall 0.5
Confusion matrix [[1444  0]
 [ 89  0]]
```

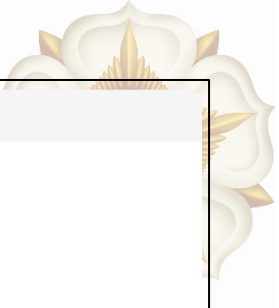
Gambar 3.4.22 Output Performa Model

- Visualisasi *confusion matrix*



Gambar 3.4.23 Confusion Matrix

c. *Decision Tree*



```

model=DecisionTreeClassifier(criterion="entropy")
model.fit(X_train, y_train)

# membuat prediksi
y_pred = model.predict(X_test)

#menghitung performa model, dengan accuracy dll
print('Accuracy ',accuracy_score(y_test, y_pred))
print('Precision ',precision_score(y_test, y_pred, average='macro'))
print('Recall ',recall_score(y_test, y_pred, average='macro'))
print('Confusion matrix ', confusion_matrix(y_test, y_pred))

# plot_confusion_matrix(model, X_test, y_test, cmap=plt.cm.Blues)
cm = confusion_matrix(y_test, y_pred)
disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=np.unique(y_test))

# plt.show()
disp.plot(cmap=plt.cm.Blues)

```

Gambar 3.4.24 Kode untuk Melatih Model Decision Tree

- Melatih model

Model *Decision Tree* dibentuk menggunakan kriteria pemilihan *split entropy*, yang mengukur tingkat ketidakaturan data pada setiap *node*. Semakin rendah nilai *entropy*, semakin baik kualitas pemisahan yang dilakukan. Data *training* (X_{train} dan y_{train}) digunakan untuk membangun struktur pohon keputusan yang optimal berdasarkan pola data.

- Membuat prediksi

Model yang telah terbentuk kemudian digunakan untuk memprediksi kelas dari data testing (X_{test}). Prediksi dilakukan dengan menelusuri jalur pada pohon keputusan, mulai dari *root* hingga mencapai *node* daun yang menentukan kelas akhir.

- Mengukur performa model

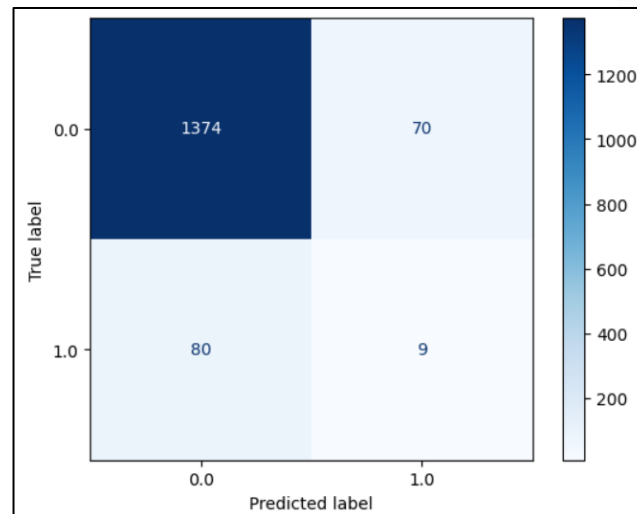
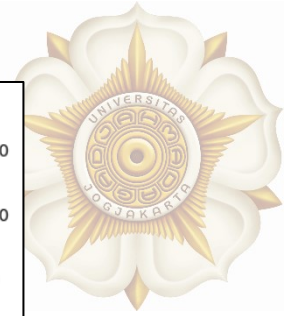
```

Accuracy 0.9021526418786693
Precision 0.5294517089478175
Recall 0.5263235706059946
Confusion matrix [[1374  70]
 [ 80   9]]

```

Gambar 3.4.25 Output Performa Model

- Visualisasi *confusion matrix*



Gambar 3.4.26 Confusion Matrix

d. Random Forest

```

model=RandomForestClassifier()
model.fit(X_train, y_train)

# membuat prediksi
y_pred = model.predict(X_test)

#menghitung performa model, dengan accuracy dll
print('Accuracy ',accuracy_score(y_test, y_pred))
print('Precision ',precision_score(y_test, y_pred, average='macro'))
print('Recall ',recall_score(y_test, y_pred, average='macro'))
print('Confusion matrix ', confusion_matrix(y_test, y_pred))

# plot_confusion_matrix(model, X_test, y_test, cmap=plt.cm.Blues)
cm = confusion_matrix(y_test, y_pred)
disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=np.unique(y_test))

# plt.show()
disp.plot(cmap=plt.cm.Blues)

```

Gambar 3.4.27 Kode untuk Melatih Model Random Forest

- Melatih model

Model *Random Forest* dibentuk dengan menggunakan parameter *default*, yang secara otomatis membuat sejumlah pohon keputusan dan melatihnya pada subset acak dari dataset. Algoritma ini bekerja berdasarkan prinsip *bagging*, yaitu menggabungkan hasil prediksi dari banyak pohon untuk mengurangi varians dan meningkatkan akurasi. Data *training* (X_{train} dan y_{train}) digunakan untuk membangun *forest* tersebut.



- Membuat prediksi

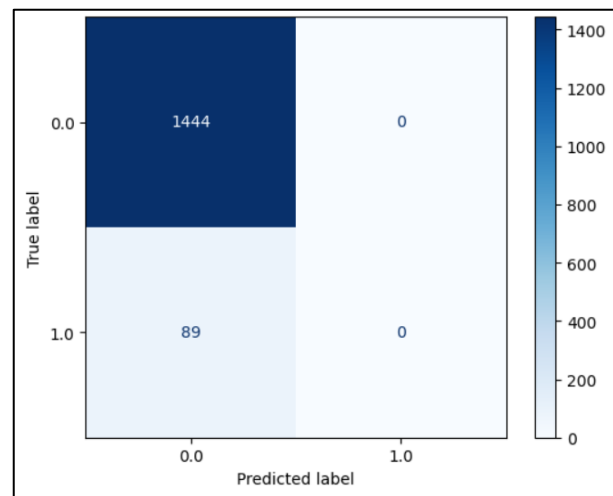
Model yang telah dilatih kemudian digunakan untuk memprediksi kelas data testing (X_{test}). Prediksi akhir ditentukan melalui proses voting mayoritas dari seluruh pohon yang ada dalam *ensemble*.

- Mengukur performa model

```
Accuracy 0.9419439008480104
Precision 0.4709719504240052
Recall 0.5
Confusion matrix [[1444  0]
 [ 89  0]]
```

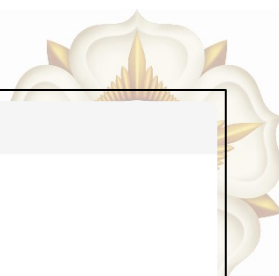
Gambar 3.4.28 Output Performa Model

- Visualisasi *confusion matrix*



Gambar 3.4.29 Confusion Matrix

e. Ada Boost



```

model=AdaBoostClassifier()
model.fit(X_train, y_train)

# membuat prediksi
y_pred = model.predict(X_test)

#menghitung performa model, dengan accuracy dll
print('Accuracy ',accuracy_score(y_test, y_pred))
print('Precision ',precision_score(y_test, y_pred, average='macro'))
print('Recall ',recall_score(y_test, y_pred, average='macro'))
print('Confusion matrix ', confusion_matrix(y_test, y_pred))

# plot_confusion_matrix(model, X_test, y_test, cmap=plt.cm.Blues)
cm = confusion_matrix(y_test, y_pred)
disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=np.unique(y_test))

# plt.show()
disp.plot(cmap=plt.cm.Blues)

```

Gambar 3.4.30 Kode untuk Melatih Model Ada Boost

- Melatih model

Model *AdaBoost* dibentuk menggunakan parameter *default*, di mana algoritma akan melatih beberapa *weak learners* (biasanya *decision tree* berukuran kecil) secara berurutan. Setiap *weak learner* diberikan bobot berdasarkan tingkat kesalahannya, sehingga model berikutnya lebih fokus pada sampel-sampel yang sulit diprediksi. Proses ini meningkatkan akurasi keseluruhan model. Data latih (*X_train* dan *y_train*) digunakan sebagai dasar pembelajaran.

- Melakukan prediksi

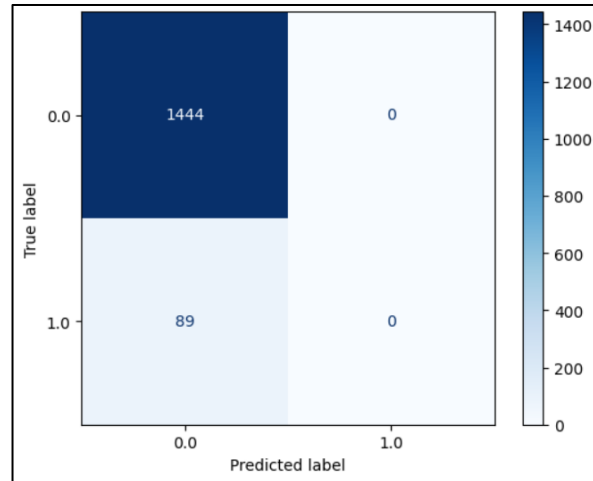
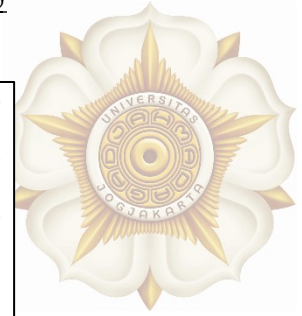
Setelah proses pelatihan selesai, model digunakan untuk memprediksi kelas data testing (*X_test*). Prediksi akhir merupakan kombinasi berbobot dari seluruh *weak learners* yang telah dilatih sebelumnya.

- Mengukur performa model

Accuracy	0.9419439008480104
Precision	0.4709719504240052
Recall	0.5
Confusion matrix	[[1444 0]
	[89 0]]

Gambar 3.4.31 Output Performa Model

- Visualisasi *confusion matrix*



Gambar 3.4.32 Confusion Matrix

3.5. TUGAS & ANALISIS

Dataset: <https://www.kaggle.com/datasets/bhavikjikadara/loan-status-prediction>

1. Jelaskan apa tujuan penggunaan dataset ini?
2. Definisikan atribut yang menjadi input dan output nya?
3. Silahkan membuat model klasifikasi *logistic regression*, KNN, *Decision Tree*, dan *Random Forest* untuk kasus dataset diatas!
4. Buatlah tabel yang menjelaskan performa dari model *machine learning* untuk kasus dataset diatas! Kolom pertama "model", kolom selanjutnya "*accuracy, precision, recall*"
5. Jelaskan dari hasil eksperimen di atas, model mana yang paling baik? Jelaskan alasan anda!

3.6. REFERENSI

<https://www.python.org/doc/>

<https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.html>

<https://numpy.org/>

Han, J., Pei, J., & Tong, H. (2022). Data Mining: Concepts and Techniques (4th ed.). Morgan Kaufmann/Elsevier.